Trabajo Práctico 3 – Ejercicios extra

SISTEMAS OPERATIVOS

Tracks

1. **Páginas bajo demanda y memoria residente (7/10)** **4**

2. **`mmap()`, permisos y regiones anónimas (7/10)** **5**

3. **Intercambio y presión de memoria (2/10)** **6**

4. **Bibliotecas compartidas y resolución de símbolos (5/10)** **7**

5. **Inspección de un plugin VST (4/10)** **8**

6. **Diagnóstico de errores de memoria (6/10)** **9**

Estos tracks amplían el práctico de memoria con experimentos comparables. Registrá arquitectura, versión de `gcc` y opciones de compilación; las direcciones concretas no son portables, así que explicá patrones, permisos, tamaños y relaciones.

Los tracks son independientes entre sí: elegí uno después de completar el práctico regular y no necesitás resolver otro track para empezar. Las cajas **EXTRA** proponen una extensión opcional del track; las cajas **BONUS – POR PUNTOS** plantean una implementación más profunda. El práctico regular no requiere un informe; para obtener puntos de bonus, entregá una explicación breve con capturas y quedate disponible para una demostración.

1. Páginas bajo demanda y memoria residente (7/10)

TRACK – 7/10

Descripción: diferenciar el espacio virtual reservado de las páginas tocadas y residentes.

Escribí un programa `demanda.c` que demuestre la paginación bajo demanda (*demand paging*). Observá los campos `Vm*` de `/proc/<pid>/status`; provocá accesos a páginas nuevas y registrá los fallos `minflt`/`majflt` de `/proc/<pid>/stat`. Relacioná esos fallos con los cambios que observes en `/proc/<pid>/smaps_rollup` y explicá el resultado.

```
$ gcc -Wall -Wextra -O0 -g demanda.c -o demanda
$ ./demanda &
$ pid=$!
$ grep '^Vm' /proc/$pid/status
$ awk '{printf "minflt=%s majflt=%s\n", $10, $12}' /proc/$pid/stat
$ cat /proc/$pid/smaps_rollup
$ wait $pid
```

EXTRA

Formulá una hipótesis antes de tocar las páginas. Explicá por qué `malloc()` puede aumentar `VmSize` sin aumentar inmediatamente `VmRSS`, y qué evento provoca la primera escritura en cada página.

2. `mmap()`, permisos y regiones anónimas (7/10)

TRACK – 7/10

Descripción: crear y modificar mapeos explícitos para relacionar llamadas al sistema, permisos y entradas de `maps`.

1. Reservar una región anónima con `mmap()` usando `PROT_READ` | `PROT_WRITE` y `MAP_PRIVATE` | `MAP_ANONYMOUS`.
2. Imprimir la dirección, escribir en la primera y última página y localizar la región en `/proc/<pid>/maps`.
3. Usar `mprotect()` para quitar escritura y volver a inspeccionar.
4. Mapear un archivo real con `MAP_PRIVATE`, modificar un byte mediante el mapeo y comprobar que el archivo no cambia. Explicá qué significa que la modificación sea privada y contrastá, opcionalmente, con `MAP_SHARED`.
5. Liberar las regiones con `munmap()` y comprobar que desaparecen.

```
void *region = mmap(NULL, 4 * 4096,
                    PROT_READ | PROT_WRITE,
                    MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
if (region == MAP_FAILED)
{
    perror("mmap");
    return 1;
}

if (mprotect(region, 4 * 4096, PROT_READ) == -1)
    perror("mprotect");
```

Usá `pmap -x <pid>` como vista alternativa y compará sus columnas con `maps`. Investigá por qué una región privada puede compartir páginas físicamente mediante *copy-on-write*.

3. Intercambio y presión de memoria (2/10)

TRACK – 2/10

Descripción: activar temporalmente un archivo de intercambio (*swapfile*) y observar cómo aparece en las herramientas del sistema.

Trabajá en una máquina de prueba y usá un archivo descartable. Necesitás privilegios para ejecutar `swapon`; no modifiques `/etc/fstab`, no uses `swapoff -a` y no desactives particiones o archivos de intercambio que ya estuvieran activos. El archivo debe estar en un sistema de archivos que admita swapfiles.

Desde un directorio con espacio libre, creá y activá un archivo de 256 MiB:

```
$ swapon --show
$ df -h .
$ fallocate -l 256M swapfile-demo
$ chmod 600 swapfile-demo
$ sudo mkswap swapfile-demo
$ sudo swapon swapfile-demo
$ swapon --show
$ cat /proc/swaps
$ free -h
```

Si `fallocate` no está disponible o el sistema de archivos rechaza el archivo, investigá una alternativa que cree un archivo completamente asignado, por ejemplo:

```
$ dd if=/dev/zero of=swapfile-demo bs=1M count=256 status=progress
```

Mientras ejecutás una carga de memoria moderada, observá `vmstat 1`, `free -h` y `/proc/meminfo`. Distinguí entre tener swap habilitado y que el núcleo efectivamente mueva páginas hacia él; no es necesario agotar la memoria de la máquina para realizar la observación.

Al terminar, desactivá solamente el archivo de la práctica, comprobá que ya no aparece y recién después borrálo:

```
$ sudo swapoff swapfile-demo
$ swapon --show
$ rm swapfile-demo
```

Si `swapoff` falla porque el archivo todavía está en uso, no lo borres.

4. Bibliotecas compartidas y resolución de símbolos (5/10)

TRACK – 5/10

Descripción: investigar cómo el cargador encuentra una biblioteca compartida y cuándo resuelve un símbolo del ejecutable.

Partí de los archivos `saludo.c` y `main.c` del TP3 regular: `saludo.c` define la función de la biblioteca y `main.c` la invoca. Construí una variante dinámica sin una ruta de búsqueda incrustada, para que puedas investigar cómo el cargador encuentra `libsaludo.so`:

```
$ gcc -Wall -Wextra -fPIC -shared saludo.c \
    -Wl,-soname,libsaludo.so -o libsaludo.so
$ gcc -Wall -Wextra main.c -L. -lsaludo -o saludo
$ ldd ./saludo
$ readelf -dW ./saludo
$ readelf -dW ./libsaludo.so
$ readelf -rW ./saludo
$ LD_LIBRARY_PATH=. ./saludo
$ LD_DEBUG=libs,bindings LD_LIBRARY_PATH=. ./saludo 2>&1 | less
```

Prepará una segunda copia de `libsaludo.so` en un directorio diferente, con un mensaje distinguible, y compará:

```
$ LD_LIBRARY_PATH=alternativa:. ./saludo
$ LD_LIBRARY_PATH=.:alternativa ./saludo
```

Explicá qué biblioteca se elige en cada caso; usá `LD_DEBUG` para justificar la búsqueda y `readelf -dW` y `readelf -rW` para relacionarla con `SONAME`, `NEEDED` y las reubicaciones. Compará el enlace perezoso con `LD_BIND_NOW=1` y describí cuándo se resuelve `saludar`. No copies direcciones.

5. Inspección de un plugin VST (4/10)

TRACK – 4/10

Descripción: inspeccionar un plugin VST (*Virtual Studio Technology*) real como una biblioteca compartida, sin ejecutarlo.

Elegí un plugin VST o VST3 para Linux de una fuente confiable. No hace falta instalar un DAW (*Digital Audio Workstation*) ni producir audio. Si el plugin tiene formato `.vst3`, localizá dentro del paquete la biblioteca correspondiente a Linux; si es un archivo `.so`, usalo directamente.

Inspeccioná la biblioteca con las herramientas del track anterior:

1. Usá `file` para identificar el formato y la arquitectura.
2. Usá `ldd` para listar sus dependencias y comprobar si falta alguna.
3. Usá `readelf -hW` y `readelf -dW` para observar el tipo de archivo, el intérprete y las bibliotecas requeridas.
4. Usá `nm -D -defined-only` para observar algunos símbolos exportados.
5. Explicá qué tendría que hacer un programa anfitrión para usar esta biblioteca y qué riesgo existe al cargar código de terceros.

Registrá el nombre y la versión del plugin, la arquitectura, sus dependencias y tres observaciones de los comandos. No copies direcciones: relacioná los resultados con la carga de bibliotecas compartidas.

BONUS – POR PUNTOS

Implementá un cargador de plugins usando `dlopen()`, `dlsym()` y `dlclose()`.
Definí una interfaz binaria mínima (ABI, *Application Binary Interface*), por ejemplo una función `int plugin_run(const char *argumento)`. Creá dos plugins `.so` que implementen esa función y un programa `runner` que reciba por línea de comandos la ruta del plugin, lo cargue con `RTLD_NOW | RTLD_LOCAL`, busque el símbolo y lo ejecute.
El programa debe informar errores de `dlopen()` y `dlsym()` mediante `dlerror()`, y cerrar la biblioteca con `dlclose()`.
Compará este programa con otro enlazado directamente contra una biblioteca mediante `-l`. Usá `readelf -dW`, `LD_DEBUG=libs,bindings` y `/proc/<pid>/maps` para explicar:

- por qué el plugin cargado con `dlopen()` no aparece como dependencia `NEEDED` inicial;
- cuándo se resuelve el símbolo;
- qué diferencia hay entre `RTLD_LOCAL` y una biblioteca enlazada normalmente;
- qué puede fallar si el plugin no existe, no exporta el símbolo o usa una interfaz incompatible;
- qué riesgos tiene cargar bibliotecas proporcionadas por terceros.

6. Diagnóstico de errores de memoria (6/10)

TRACK – 6/10

Descripción: provocar errores controlados y comparar Valgrind, AddressSanitizer y un depurador.

Usá este programa sólo en un entorno de prueba:

```c
#include <stdlib.h>

int main() {
    char *buffer = malloc(4);
    buffer[4] = 1; /* escritura fuera de limites */
    free(buffer);
    free(buffer); /* liberacion doble */
    return 0;
}
```

1. Ejecutarlo sin herramientas y observar si el error es determinista.
2. Usar `valgrind -leak-check=full ./errores`.
3. Compilar con `-fsanitize=address -fno-omit-frame-pointer -g` y repetir.
4. Ejecutarlo bajo `gdb`: detenerse en `main`, avanzar con `next` y obtener un `backtrace` cuando aparezca el error.
5. Comparar pila, ubicación y momento en que cada herramienta detiene el programa.
6. Observar las bibliotecas del runtime en `/proc/<pid>/maps` mientras el programa está detenido.

```
$ gcc -Wall -Wextra -g errores.c -o errores
$ valgrind --leak-check=full ./errores
$ gcc -Wall -Wextra -g -fsanitize=address \
    -fno-omit-frame-pointer errores.c -o errores-asan
$ ./errores-asan
```

```
$ gdb ./errores
(gdb) break main
(gdb) run
(gdb) next
(gdb) continue
(gdb) backtrace
```

BONUS – POR PUNTOS

Construí una versión corregida con tres modos de error seleccionables (desborde, uso después de `free` y fuga) y un script que ejecute cada modo con Valgrind y AddressSanitizer. Compará los diagnósticos y explicá por

qué un fallo de segmentación no necesariamente señala la línea donde terminó el proceso. Incluí capturas y dejá preparada una demostración.